ALTERNATE ASSESSMENT REPORT

on

Major Project Phase-2

Course Code: 23CS7PWPP2 | Semester: VII

ScamGuard: AI-Based Online Scam and Fraud Detection System

Submitted by

Bhavya Goyal	1BM23CS063
Priyam Thakur	1BM23CS252
Samir Chaudhary	1BM23CS294
Sana Subodh	1BM23CS296

Department of Computer Science and Engineering

Faculty In-charge (Project Guide)

Dr. Kayarvizhy N

Professor, Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

Date of Submission: [DD-MM-YYYY]

Contents

Chapter 1 Abstract **1**

Chapter 2 Objectives **2**

Chapter 3 Introduction **3**

3.1 Background 3
3.2 The ScamGuard application 3
3.3 Role of machine learning and client–server design 3
3.4 Tools and technologies used 4
3.5 Organisation of the report 4

Chapter 4 Literature Review **5**

Chapter 5 Research Gap **7**

5.1 Gaps identified in the literature 7
5.2 Selected research gap 7

Chapter 6 Problem Statement **8**

Chapter 7 Methodology and Implementation **9**

7.1 Proposed system architecture 9
7.2 Dataset: ScamGuard-IN-2026 10
7.3 Mathematical model 11
7.3.1 Problem definition 11
7.3.2 Near-duplicate campaigns 11
7.3.3 Feature extraction 11
7.3.4 Classifier 12
7.3.5 Intent evidence and fusion 12
7.4 Client–server communication model 12
7.5 Proposed algorithms 14
7.6 Implementation 15

7.7 Sub-systems and interface design 16
7.8 Testing 16
7.9 Project plan, resources and cost 17

Chapter 8 Performance Evaluation 18
8.1 Experimental setup 18
8.2 Performance metrics 19

Chapter 9 Output and Results 20
9.1 Comparative analysis 20
9.2 Validation of the proposed approach 22
9.3 Observations 23
9.4 Output screens 23

Chapter 10 Conclusion 28
10.1 Mapping to course outcomes 28

Chapter 11 References 30

Chapter 1

Abstract

SMS remains a common route for financial fraud in India. Scam messages copy bank KYC reminders, electricity disconnection notices, parcel updates and traffic challans, often in Hindi, so they look much like the genuine service messages people receive every day. The goal of this work was to build ScamGuard, a web application that tells a user whether a pasted message is a scam and explains why. We assembled ScamGuard-IN-2026, a corpus of 11,399 Indian messages drawn from user-reported smishing, real genuine SMS, 2025–26 advisory texts and synthetic messages used only for training. All 2,922 groups of reported messages were checked, and 19.5% of the retained reports turned out to be genuine. Messages were grouped into near-duplicate campaigns with MinHash so that no campaign appears in both training and testing. A logistic-regression classifier over word and character TF-IDF features is combined with interpretable intent rules and served through a FastAPI back end and a Next.js front end. On 1,874 real messages from unseen campaigns the deployed system reaches 97.1% accuracy, 98.2% precision and 97.2% recall with a 3.2% false-alarm rate, and scores a message in 3.8 ms. Training on unchecked report labels instead raises the false-alarm rate to 34.0%.

Keywords: scam detection, smishing, SMS fraud, machine learning, natural language processing, label noise, data leakage, explainable AI

Chapter 2

Objectives

The assessment aimed to achieve the following.

1. To understand how SMS scams aimed at Indian users are written, and why they are hard to separate from genuine bank, operator and government messages.
2. To build a labelled dataset of Indian scam and genuine SMS that covers current scam types and whose labels have been checked rather than taken from user reports at face value.
3. To evaluate lightweight text classifiers under a protocol that keeps near-duplicate messages of one scam campaign out of the test set when they appear in training.
4. To combine the best classifier with interpretable intent rules that highlight the exact phrases behind each verdict, and to measure honestly whether the rules also improve accuracy.
5. To implement the detector as a client–server web application with a quick check, a full analysis, safety tools, history and analytics, and to deploy it on the cloud.
6. To analyse the performance of the system in terms of accuracy, precision, recall, F1-score, false-alarm rate and response time, and to make every reported number reproducible from the project repository.

Chapter 3

Introduction

3.1 Background

A scam SMS rarely looks like a scam. It looks like a message from a bank asking for a quick KYC update, from an electricity board warning of disconnection tonight, from a courier holding a parcel for a small fee, or from the traffic police with a challan to pay through an attached app. Fraudsters copy the format of genuine service messages because people have learned to trust those formats. A 2025 measurement study of more than 64,000 public smishing reports found that most of the sender numbers that could be attributed to an operator belonged to Indian mobile networks [1].

A detector has to tell the two kinds of message apart even though they share most of their vocabulary: both mention OTPs, accounts, KYC, links and deadlines. A tool that flags a real bank alert as a scam does damage of its own, because it teaches the user to ignore warnings. False alarms therefore matter as much as missed scams.

3.2 The ScamGuard application

ScamGuard analyses a single message pasted by the user. It returns a verdict, a risk score and the reasons behind them: the phrases that make the message risky, the stage of the scam each phrase belongs to (hook, pressure, request or cash-out), what a genuine message from the same organisation would do instead, and what the user should do next. The application also accepts URLs, e-mails, screenshots, PDFs and QR codes, and offers tools for the situations around a scam message, such as a UPI payment request or a suspicious file.

3.3 Role of machine learning and client–server design

The core of the system is a text classifier. Text classification turns each message into a numerical vector and learns, from labelled examples, which patterns separate scams from genuine messages. Its quality depends on the training data more than on the choice of algorithm, which is why a large part of this work went into building and checking the

dataset. The classifier is combined with hand-written intent rules that recognise well-known scam tactics and give phrase-level explanations.

The detector is delivered as a client–server application. The browser client sends the message to a REST API over HTTPS. The server cleans the text, runs the classifier and the rules, stores the result for signed-in users and returns a JSON response, which the client renders as a verdict with highlighted phrases. Rate limits, signed access tokens and request-size limits protect the service.

3.4 Tools and technologies used

- **Languages:** Python 3.13 for the dataset, model and back end; TypeScript for the front end.
- **Machine learning:** scikit-learn [19], NumPy and pandas; datasketch for MinHash; Matplotlib for figures.
- **Back end:** FastAPI, Uvicorn, SQLAlchemy with PostgreSQL (SQLite locally), SlowAPI for rate limiting, JSON Web Tokens for authentication.
- **Front end:** Next.js 14, React 18 and Tailwind CSS, with English and Hindi interfaces.
- **Deployment and testing:** Render cloud hosting, GitHub Actions for continuous integration and a daily check of the live site, and 818 automated tests.

3.5 Organisation of the report

Chapter 4 reviews the literature and Chapter 5 identifies the research gap, which Chapter 6 turns into a problem statement. Chapter 7 describes the architecture, mathematical model, communication model, algorithms and implementation. Chapter 8 gives the experimental setup and metrics, Chapter 9 the results and outputs, and Chapter 10 the conclusions.

Chapter 4

Literature Review

We reviewed sixteen peer-reviewed papers and recent preprints on SMS spam filtering, smishing detection, report-based measurement and machine-learning methodology. Each one was checked against its publisher page, DOI record or arXiv entry, and the results quoted are those stated by the authors.

SMS spam filtering. Almeida, Gómez Hidalgo and Yamakami released the SMS Spam Collection, 5,574 English messages of which 747 are spam [4]. In a follow-up study a linear SVM reached 97.64% accuracy while blocking only 0.18% of legitimate messages [5]. Roy, Singh and Banerjee reported up to 99.44% accuracy with CNN and LSTM models on the same data [6]. Ahmadi *et al.* compared six classifiers with TF-IDF and bag-of-words features and reported accuracies between 0.91 and 0.95 [13], and Labonne and Moran showed that a fine-tuned Flan-T5 model beats classical baselines on e-mail spam when labelled data is scarce [12]. These studies target promotional spam rather than fraud.

Smishing detection. SmiDCA selects content features for machine-learning anti-smishing classifiers [8]. Mishra and Soni combined content analysis with the behaviour of embedded URLs and reported 96.29% accuracy [7]. Goel *et al.* normalised slang in 362 smishing messages and trained Naive Bayes, reaching 96.2% accuracy with a 3.87% false-positive rate [9].

Report-based corpora. Timko and Rahman released 1,062 validated Smishtank reports with URL and sender metadata [10]. Agarwal *et al.* mined five public forums from 2017 to 2024 and released 33,869 labelled messages, noting that user reports carry no verification [1]. The same group found that 35.12% of unique reported texts in 1.2 million UK operator reports were spam rather than scams [2]. MOZ-Smishing provides labelled genuine and fraudulent mobile-money messages for Mozambique [11]. A 2026 systematic review of 119 smishing papers found datasets "often limited in scale, diversity, and temporal coverage" [3].

Methodology. Confident learning locates label errors from disagreements between cross-validated predictions and given labels [14]. Lee *et al.* showed that near-duplicates between training and validation data inflate evaluation [15], and Kapoor and Narayanan list such duplicates among the most common leakage errors in applied machine learning [16].

MinHash [17] and TF-IDF weighting [18] are the standard tools for finding near-duplicates and for representing text. Table 4.1 summarises the reviewed work.

Table 4.1: Summary of the literature review

No.	Work	Technique	Result	Limitation for Indian scams
1	Almeida *et al.*, 2011 [4]	SMS spam corpus and baselines	5,574 labelled messages	Promotional spam, no Indian messages
2	Almeida *et al.*, 2013 [5]	15+ classifiers, tokenisers	SVM 97.64% accuracy	Same corpus; no scams of current types
3	Roy *et al.*, 2020 [6]	CNN, LSTM	99.44% accuracy	No cross-domain test
4	Sonowal, Kuppusamy, 2018 [8]	Feature selection, ML classifiers	Anti-smishing model	Small English datasets
5	Mishra, Soni, 2020 [7]	Content + URL behaviour	96.29% accuracy	Misses scams without links
6	Goel *et al.*, 2024 [9]	Slang normalisation, Naive Bayes	96.2% acc., FPR 3.87%	Only 362 smishing messages
7	Timko, Rahman, 2024 [10]	Validated report collection	1,062 reports	No genuine class
8	Agarwal *et al.*, 2025 [1]	GPT-4o labelling of reports	33,869 messages, 66 languages	Reports unverified; no genuine class
9	Agarwal *et al.*, 2026 [2]	Analysis of operator reports	35.12% of reports are spam	UK data; no classifier
10	Ali *et al.*, 2025 [11]	LLM in-context learning	Mobile-money benchmark	Mozambique context
11	Labonne, Moran, 2023 [12]	Fine-tuned Flan-T5	Beats classical baselines	E-mail, not SMS
12	Ahmadi *et al.*, 2025 [13]	TF-IDF, six classifiers	0.91–0.95 accuracy	Tables and text disagree
13	Pritom *et al.*, 2026 [3]	Systematic review	119 papers, 12 datasets	Identifies the dataset gap
14	Northcutt *et al.*, 2021 [14]	Confident learning	Finds label errors	General method
15	Lee *et al.*, 2022 [15]	Near-duplicate removal	Overlap inflates results	Language-model corpora
16	Kapoor, Narayanan, 2023 [16]	Survey of leakage errors	Duplicates are common	General method

Chapter 5

Research Gap

5.1 Gaps identified in the literature

1. **No verified genuine Indian class.** Classical corpora contain no Indian messages, and report-based corpora contain only reported messages. No public resource pairs Indian scams with Indian genuine messages under checked labels.
2. **Report labels taken at face value.** Report-based corpora are used as if every report were a scam, although their authors warn that reports are unverified [1, 2].
3. **Evaluation ignores campaigns.** One scam template is sent to many people with small edits. Random splits put copies of the same template on both sides, and none of the surveyed SMS studies controls for this [15, 16].
4. **Old scam types.** Most data predates 2025, so digital-arrest calls, e-challan APKs, task-based jobs and similar current scams are rare or absent [3].
5. **Rules not evaluated independently.** Hybrid systems add hand-written rules, but their benefit is usually measured on messages chosen by the same developers.
6. **No explanation for the user.** Most detectors return a bare label and do not show which part of the message is the trap.

5.2 Selected research gap

Gaps 1 to 3 share one root: there is no Indian scam-versus-genuine SMS benchmark with checked labels and a leakage-free split. Without it, every reported accuracy for Indian users is uncertain, because the labels may be wrong and the test set may contain copies of training messages. This work narrows down to this single gap:

The absence of an audited, campaign-aware Indian SMS dataset with a verified genuine class, and of a detector evaluated on it.

The other gaps are handled in support of it: synthetic 2025–26 messages, independent tests of the rules, and phrase-level explanations.

Chapter 6

Problem Statement

To design, implement and evaluate an AI-based system that classifies an Indian SMS, in English or Hindi, as a scam or a genuine message with a low false-alarm rate, using a dataset whose labels have been audited and whose train–test split keeps every scam campaign on one side; and to deliver the system as a deployed client–server web application that explains each verdict at phrase level.

The statement implies the following requirements.

- **Input:** one message as the user would paste it, without sender metadata.
- **Output:** a verdict (scam or legitimate), a risk score between 0 and 1, the phrases that drove the verdict and recommended next steps.
- **Definition of a scam:** a message that tries to obtain money, a secret such as an OTP or PIN, personal data, an app installation or a call to a private number by deception. Genuine advertisements are legitimate.
- **Constraints:** the model must run on a free cloud instance and answer within milliseconds; personal data in contributed messages must be masked.
- **Success criteria:** high recall on scams together with a low false-alarm rate on genuine messages, measured on real messages from campaigns never seen in training.

Chapter 7

Methodology and Implementation

7.1 Proposed system architecture

Figure 7.1 shows the architecture. Training runs offline on ScamGuard-IN-2026 and stores the trained vectorizer and model in a versioned registry. At run time the web client sends a message to the FastAPI back end, which cleans and tokenises the text, computes the classifier probability, applies the intent and protective rules, fuses them into a risk score and builds the explanation. Results of signed-in users are stored in the database for history and analytics. A retrieval step matches the message against 52 documented scam patterns, and an optional AI copilot answers follow-up questions.

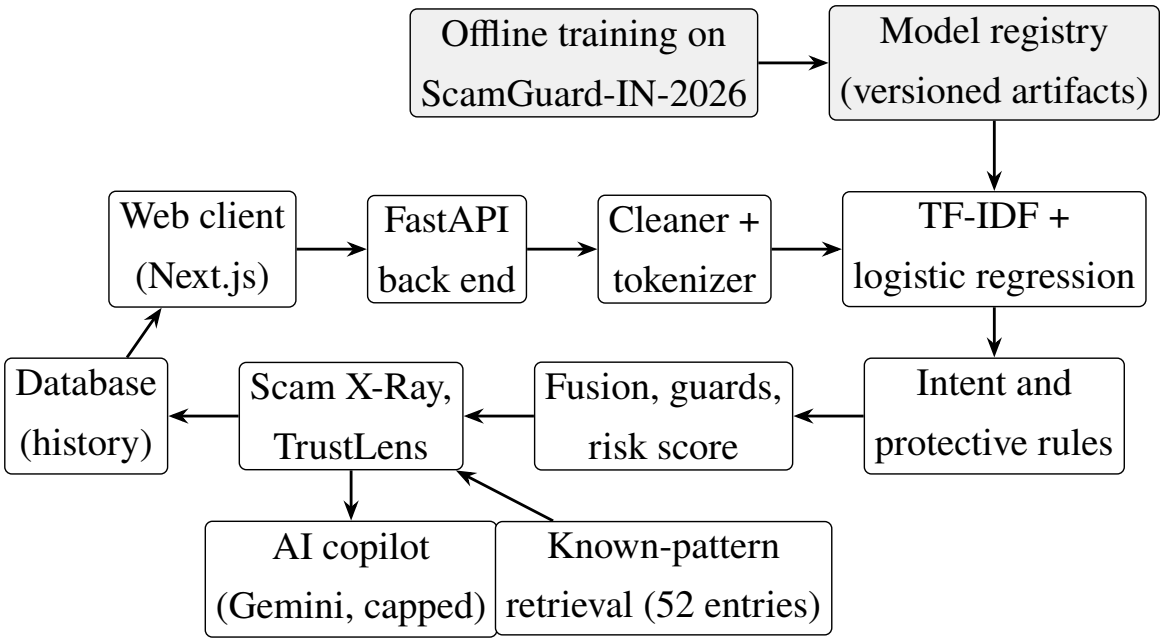


Figure 7.1: System architecture of ScamGuard. Shaded parts run offline.

7.2 Dataset: ScamGuard-IN-2026

Table 7.1 lists the sources. **R1** is the India-relevant part of the IMC 2025 smishing collection [1]: English or Hindi messages whose sender network is Indian or whose text names an Indian bank, payment app, identity document, operator or currency. **R2** contains Indian personal, work and promotional messages. **R3** holds twelve messages from 2025–26 advisories and news, stored with source and date. **S** covers scam types that are new or rare in the real data, such as digital-arrest calls, e-challan and wedding-invitation APKs, task-based jobs and Hindi electricity threats, together with genuine Indian formats such as bank alerts and OTP messages. Synthetic messages are never used for testing.

Table 7.1: Sources of ScamGuard-IN-2026

Source	Genuine	Scam	Use
R1: IMC 2025 reports, India subset (audited)	963	3,987	train, development, test
R2: India Spam SMS collection (audited)	1,694	6	train, development, test
R3: 2025–26 advisories and news	1	11	test only
S: Synthetic 2025–26 templates	2,950	1,787	training only
Total	**5,608**	**5,791**	**11,399 messages**

Label audit. Reported messages are not automatically scams. All 2,922 groups of reported messages were checked: 1,253 were confirmed by rule because they carried a strong fraud cue and no genuine cue, and the other 1,669 were read one by one (583 scam, 867 genuine, 219 unclear and dropped). A cross-validated model then flagged 44 rows whose label it strongly contradicted; re-reading them gave 19 corrections and 7 removals. After the audit, 963 of the 4,950 retained R1 rows (19.5%) are genuine.

Campaigns and splits. The 6,662 real messages form 4,085 near-duplicate campaigns; the largest has 128 messages. Each campaign goes entirely to training (60%), development (15%) or test (25%), which gives the counts in Table 7.2.

Table 7.2: Dataset splits

Split	Genuine	Scam	Content
Train	4,581	4,064	real and synthetic
Development	361	507	real only
Test	665	1,209	real only, unseen campaigns
2025–26 advisory test	1	11	real advisory texts

7.3 Mathematical model

7.3.1 Problem definition

For a message x the system computes a risk score $s(x) \in [0, 1]$ and predicts

$$\hat{y} = \begin{cases} 1 \text{ (scam)} & \text{if } s(x) \geq 0.5, \\ 0 \text{ (legitimate)} & \text{otherwise.} \end{cases} \tag{7.1}$$

7.3.2 Near-duplicate campaigns

Each message is turned into a set A of word 3-shingles after digits and links are replaced. Two messages belong to the same campaign when their Jaccard similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{7.2}$$

is at least 0.6. MinHash estimates J quickly because, for a random hash function h, $\Pr\left[\min h(A) = \min h(B)\right] = J(A, B)$ [17].

7.3.3 Feature extraction

For a term t in message d, with N training messages and document frequency $\mathrm{df}(t)$, the TF-IDF weight is

$$w_{t,d} = \left(1 + \log \mathrm{tf}_{t,d}\right) \cdot \left(\log \frac{1 + N}{1 + \mathrm{df}(t)} + 1\right), \tag{7.3}$$

and each vector is normalised to unit length. Word unigrams and bigrams (up to 30,000) and character 2–5-grams (up to 60,000) are concatenated into $\phi(x)$, with 35,148 features after pruning. Character n-grams let the model recognise spellings such as *K.Y.C* or *bl0cked*.

7.3.4 Classifier

Logistic regression estimates

$$p(x) = \sigma\big(\mathbf{w}^{\top}\phi(x) + b\big) = \frac{1}{1 + e^{-(\mathbf{w}^{\top}\phi(x)+b)}}, \tag{7.4}$$

by minimising the class-weighted, ℓ_2-regularised log-loss

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i}\Big[- y_i \log p(x_i) - (1 - y_i) \log\big(1 - p(x_i)\big)\Big], \tag{7.5}$$

with $C = 16$ and class weights $\alpha_c = n/(2n_c)$.

7.3.5 Intent evidence and fusion

Thirty-one patterns detect 26 scam intents with weights w_i and ten protective signals with weights v_j. With $E^{+} = \sum_{i \in I(x)} w_i$ and $E^{-} = \sum_{j \in P(x)} v_j$, the final score is

$$s(x) = \sigma\Big(\operatorname{clip}\big(\operatorname{logit} p(x), -4, 4\big) + 5\,(E^{+} - E^{-})\Big), \tag{7.6}$$

capped at 0.15 when the message has the shape of a genuine transaction alert or service notice without any blocking cue such as an install request or a payment demand.

7.4 Client–server communication model

The client and server communicate over HTTPS with JSON request and response bodies. Figure 7.2 shows the sequence for one analysis.

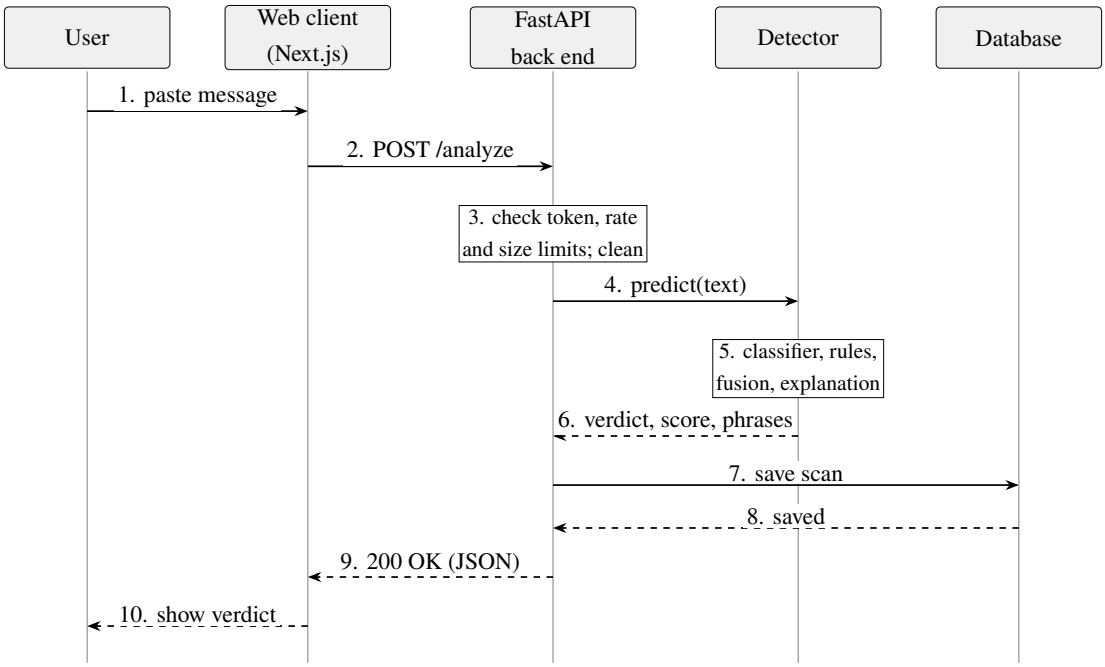


Figure 7.2: Sequence diagram of one message analysis

- **Endpoints.** POST /api/v1/public/quick-check serves visitors without an account (10 requests per minute); POST /api/v1/messages/analyze serves signed-in users (30 per minute). Other routes handle authentication, history, cases, analytics, the copilot and the safety tools.
- **Authentication.** Signed access tokens with refresh tokens; passwords are stored as hashes.
- **Robustness.** Message text is limited to 4,000 characters. If a separate model service is configured, the back end calls it with a 1.5-second timeout and otherwise runs the detector in-process, so the API never waits on a dead service.

7.5 Proposed algorithms

Algorithm 1 builds the dataset and Algorithm 2 scores one message in the live service.

Algorithm 1 Building and splitting ScamGuard-IN-2026

Require: sources R1–R3, synthetic templates S, audit files

normalise anonymised entities in R1; mask personal data
group real messages into campaigns by MinHash ($J \geq 0.6$)
for each reported campaign **do**
    if strong fraud cue and no genuine cue **then**
        label scam
    else
        apply the manual decision (scam, genuine or drop)
    end if
end for
re-read rows that a cross-validated model strongly contradicts
assign each campaign to train, development or test by a salted hash
add synthetic messages to training only; remove exact test duplicates from training
return labelled dataset with split column

Algorithm 2 Scoring one message in the live service

Require: message x; trained vectorizer and model; intent rules

$t \leftarrow \textsc{Tokenize}(\textsc{Clean}(x))$
$p \leftarrow \sigma(\mathbf{w}^\top \phi(t) + b)$
find matched intents I and protective signals P
if a sentence asks for an OTP after a "never share" warning **then**
    add decoy intent
end if
$s \leftarrow \sigma(\text{clip}(\text{logit}\, p, \pm 4) + 5(E^+ - E^-))$
if genuine alert or notice with no blocking cue **then**
    $s \leftarrow \min(s, 0.15)$
end if
return verdict $\mathbf{1}[s \geq 0.5]$, p, s, highlighted phrases, explanation

7.6 Implementation

- **Dataset build.** A deterministic Python script reads the sources, applies the audit files and writes one CSV file; the same sources always give a byte-identical file.
- **Preprocessing.** The cleaner replaces e-mail addresses, URLs (including bare links such as bit.ly/x) and phone numbers with tokens, lowercases the text and keeps Devanagari characters. The same code runs in training and in the live service.
- **Training.** scikit-learn pipelines train the four candidate models; the selected logistic regression is saved with its vectorizer in the model registry (0.28 MB model, 1.29 MB vectorizer).
- **Back end.** FastAPI with SQLAlchemy models for users, scans, cases and feedback; SlowAPI rate limits; PDF report generation; OCR for screenshots.
- **Front end.** Next.js pages for the quick check, full analysis (Scam X-Ray and TrustLens), dashboard, history, analytics, safety tools and a scam simulator, in English and Hindi, with an elder mode.
- **Deployment.** Both services run on Render. GitHub Actions runs 818 automated tests on every change and checks the live site daily.

7.7 Sub-systems and interface design

Table 7.3: Sub-systems of ScamGuard and their interfaces

Sub-system	Responsibility	Interface
Dataset builder	Collect sources, audit labels, group campaigns, split	Python script; writes one CSV file
Training pipeline	Train and compare models; register the selected one	Model registry (vectorizer and model files)
Detector	Clean, classify, apply rules, fuse, explain	Python function predict(text)
Back end (API)	Authentication, rate limits, storage, reports	REST over HTTPS, JSON
Front end	Quick check, analysis, dashboard, tools, Hindi	Browser; calls the REST API
Copilot	Answers follow-up questions about a scan	REST endpoint; external LLM API with a daily cap
Monitoring	Daily check of the live site	GitHub Actions workflow

7.8 Testing

Testing was done at three levels.

- **Unit and integration tests.** 818 automated back-end tests cover preprocessing, the feature extractor, the model registry, the detector, the rules and every API route, and 14 front-end tests cover the main components. All of them pass.
- **Continuous integration.** GitHub Actions runs the back-end and front-end tests on every change before deployment.
- **Live checks.** A scheduled workflow sends known scam and genuine messages to the deployed site every day and checks the verdicts, history and analytics pages.

Model validation (held-out test set, confidence intervals and significance tests) is described in Chapters 8 and 9.

7.9 Project plan, resources and cost

Figure 7.3 shows the Phase-2 implementation timeline taken from the commit history of the project repository. Problem study and literature review were done in Phase 1, before this repository was created.

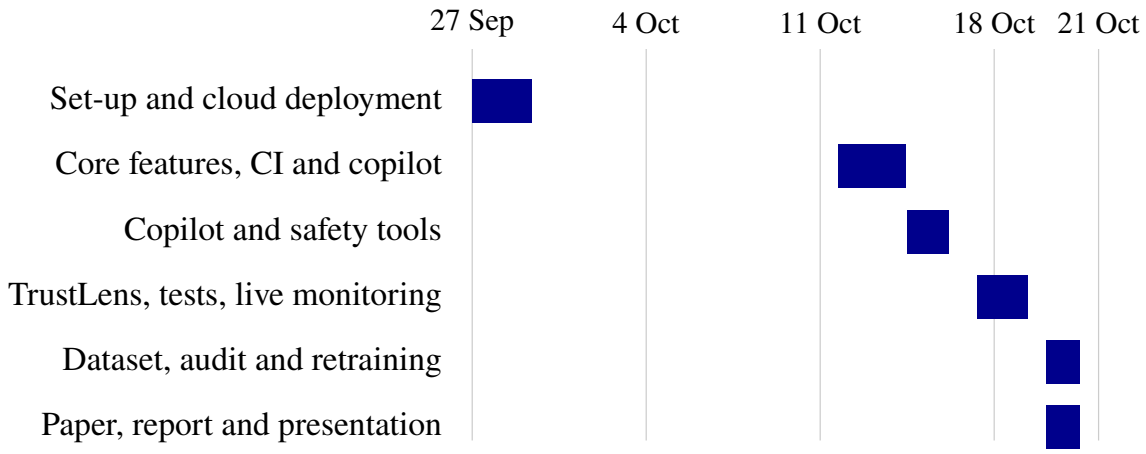


Figure 7.3: Phase-2 implementation timeline (from commit dates, 27 September to 20 October 2026)

Resources. Hardware: the team's own laptops for development and a two-core cloud instance for serving; no GPU is needed. Software: Python, scikit-learn, FastAPI, PostgreSQL, Next.js and GitHub Actions, all open source; the public datasets are used under CC BY 4.0 and MIT licences.

Cost. The application runs on Render's free tier and the copilot on a free API tier with a daily cap, so the recurring cost of the prototype is zero. A deployment for many users would need a paid always-on instance and a managed database, priced by the provider's current plans.

Chapter 8

Performance Evaluation

8.1 Experimental setup

Table 8.1: Experimental setup

Item	**Setting**
Hardware	Two-core Intel Xeon at 2.8 GHz
Software	Python 3.13, scikit-learn 1.7.2, NumPy 2.2.6, pandas 2.3.3
Training data	Training and development splits together, 9,513 messages, after model selection on the development split
Test data	1,874 real messages (665 genuine, 1,209 scam) from campaigns never seen in training; 12 advisory messages from 2025–26; 259 team-written challenge messages, used only as a development check
Candidate models	Complement Naive Bayes ($\alpha = 0.3$), logistic regression ($C = 16$), linear SVM ($C = 0.5$, calibrated), random forest (300 trees); word, character and combined features
Baselines	Training on unchecked report labels; random row-level split; classifier without rules
Reproducibility	Random seeds 2026 and 42; one script writes every result to a JSON file

8.2 Performance metrics

With scam as the positive class, a true positive (TP) is a scam that is flagged, a false positive (FP) is a genuine message that is flagged, and a false negative (FN) is a scam that passes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{8.1}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{8.2}$$

$$\text{False-alarm rate} = \frac{FP}{FP + TN}. \tag{8.3}$$

We also report the area under the ROC curve (ROC-AUC), 95% confidence intervals from 1,000 bootstrap resamples of the test set, McNemar's exact test for paired comparisons, and the response time per message measured on the hardware above.

Chapter 9

Output and Results

9.1 Comparative analysis

Table 9.1 and Figure 9.1 give the main result. The deployed system catches 1,175 of the 1,209 test scams and passes 644 of the 665 genuine messages.

Table 9.1: Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate)

System	Acc.	Prec.	Rec.	F1	FAR	AUC
Classifier only	97.5	98.6	97.6	98.1	2.6	0.997
Deployed system (hand rules)	97.1	98.2	97.2	97.7	3.2	0.995
Learned fusion	**97.8**	**98.7**	**97.8**	**98.3**	**2.3**	0.997

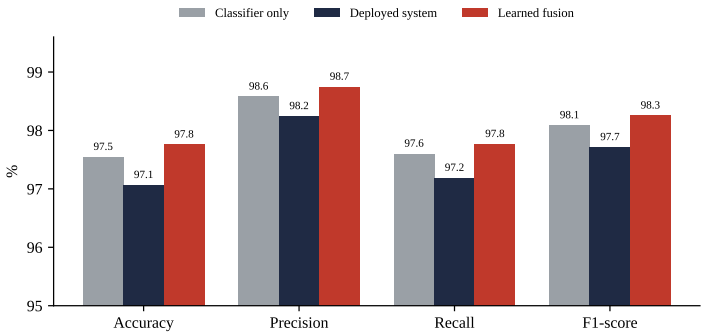


Figure 9.1: Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%)

Table 9.2 compares the candidate models. Logistic regression with word and character features has the lowest false-alarm rate among the strong models; the SVM's F1 is 0.1 points higher, but it flags 3.9% of genuine messages.

Table 9.2: Candidate models on the real test set (%)

Model	**Features**	**F1**	**False alarms**	**Challenge F1**
Naive Bayes	word + char	95.5	6.5	86.6
Logistic regression	**word + char**	**98.1**	**2.6**	90.2
Linear SVM	word + char	98.2	3.9	92.0
Random forest	word + char	97.6	3.3	73.9
Logistic regression	word only	98.0	2.7	89.9
Logistic regression	char only	96.9	4.8	86.8

Table 9.3 compares the proposed approach with the usual practice in the literature.

Table 9.3: Proposed approach against baseline practice (logistic regression, %)

Comparison	**Setting**	**F1**	**False alarms**
Labels	Report labels (baseline)	91.2	34.0
	Audited labels (proposed)	98.1	2.6
Split (real data only)	Random split (baseline)	98.3	0.8
	Campaign split (proposed)	97.7	1.4
Training data	Real only	97.7	1.4
	Real + synthetic (proposed)	98.1	2.6

The label audit has the largest effect: report labels give 99.4% recall but flag 34.0% of genuine messages. The random split looks better than the campaign split, but only because 793 of its test messages have a near-duplicate in training; it hides nearly half of the false alarms. Synthetic data costs a little precision on real messages but raises recall on the 2025–26 scam types in the challenge set from 35.6% to 82.2%.

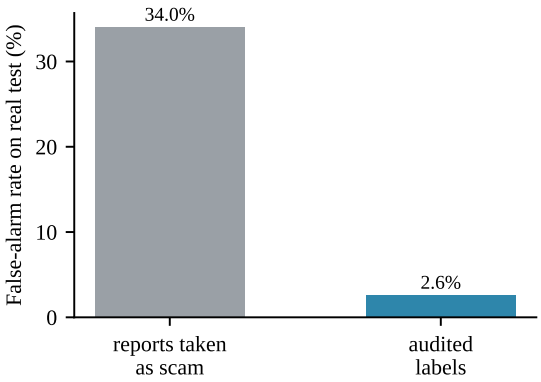


Figure 9.2: False-alarm rate with report labels and with audited labels

9.2 Validation of the proposed approach

- **Confidence intervals.** The deployed system's accuracy lies between 96.3% and 97.8%, and its false-alarm rate between 2.0% and 4.6% (95% bootstrap intervals).
- **Significance.** Audited labels beat report labels on 210 test messages and lose on 23 (McNemar $p < 10^{-30}$). Learned fusion weights are significantly better than the hand-set ones ($p = 0.002$) but not better than the classifier alone ($p = 0.34$).
- **Independent data.** On the 12 advisory messages from 2025–26 the system detects 10 of the 11 scams and passes the one genuine operator notice (F1 95.2%); the set is too small for a firm conclusion.
- **Rules.** On the team's own challenge messages the rules raise F1 from 90.2% to 95.6%, but on independent real data they lower it from 98.1% to 97.7%. The rules are kept for their phrase-level explanations, not for accuracy.

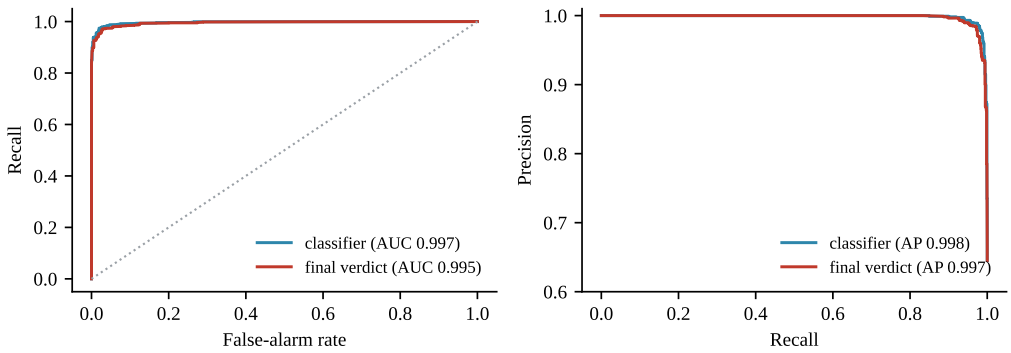


Figure 9.3: ROC and precision-recall curves on the real test set

9.3 Observations

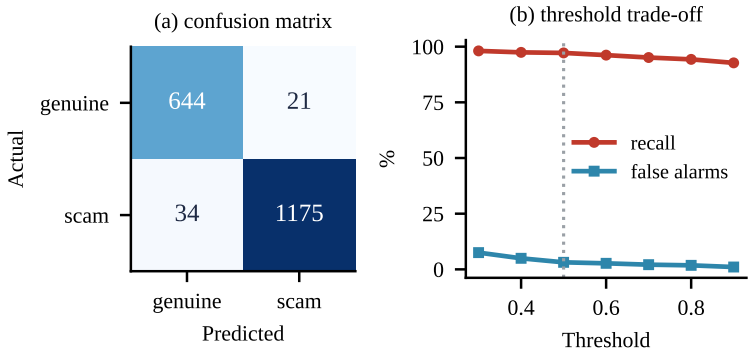


Figure 9.4: (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold

- **Errors.** At the default threshold the system misses 34 scams and flags 21 genuine messages; 15 of the 21 are reported-but-genuine bank messages that ask the user to click a link to finish a KYC or card application.
- **Threshold.** A threshold of 0.9 lowers the false-alarm rate to 0.8% while recall falls to 92.5%.
- **Scam types.** Recall is 97.7% on banking scams (895 of 916) and 92.3% on government-themed scams (24 of 26).
- **Robustness.** Obfuscating keywords such as KYC, PAN, OTP and SIM in 837 test scams lowered recall from 99.4% to 98.9% with word features and to 99.3% with word and character features.
- **Response time and size.** Scoring takes 1.7 ms per message for the classifier and 3.8 ms including the rules; training takes 2.7 s; the model and vectorizer occupy 1.6 MB.

9.4 Output screens

Figures 9.5–9.12 show the deployed application.

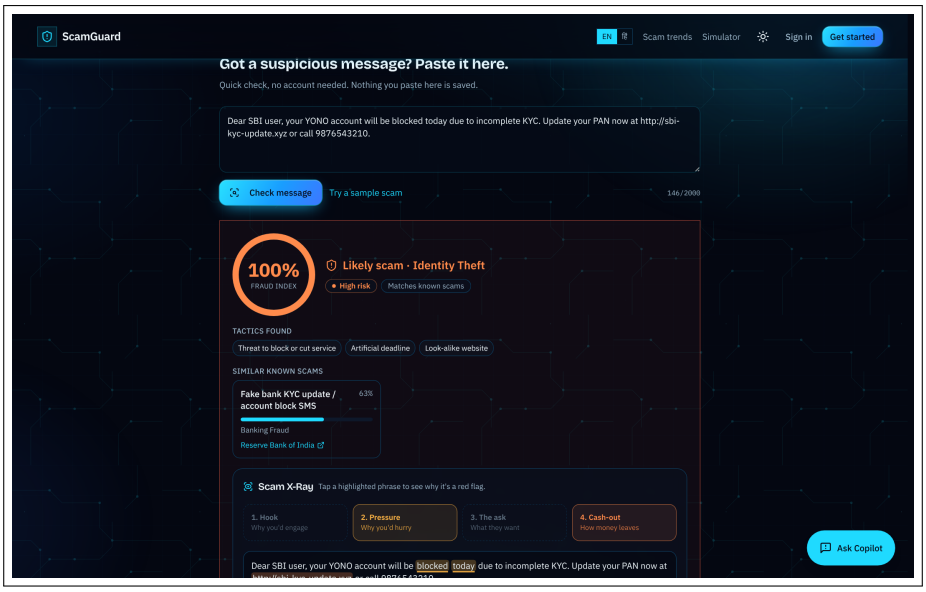


Figure 9.5: Quick check of a fake KYC message: verdict, matched tactics and Scam X-Ray

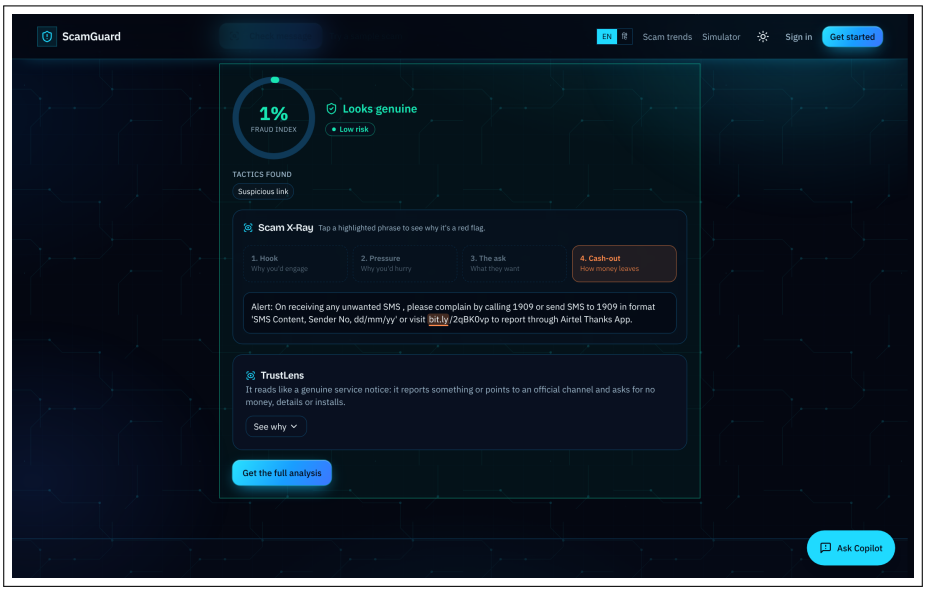


Figure 9.6: Quick check of a genuine operator notice about the 1909 helpline

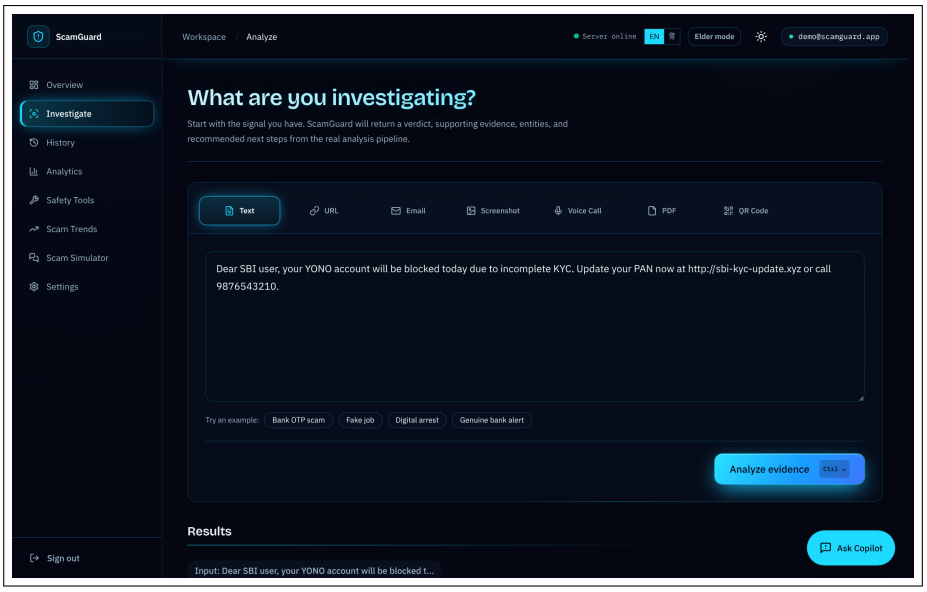


Figure 9.7: Full analysis screen for a signed-in user

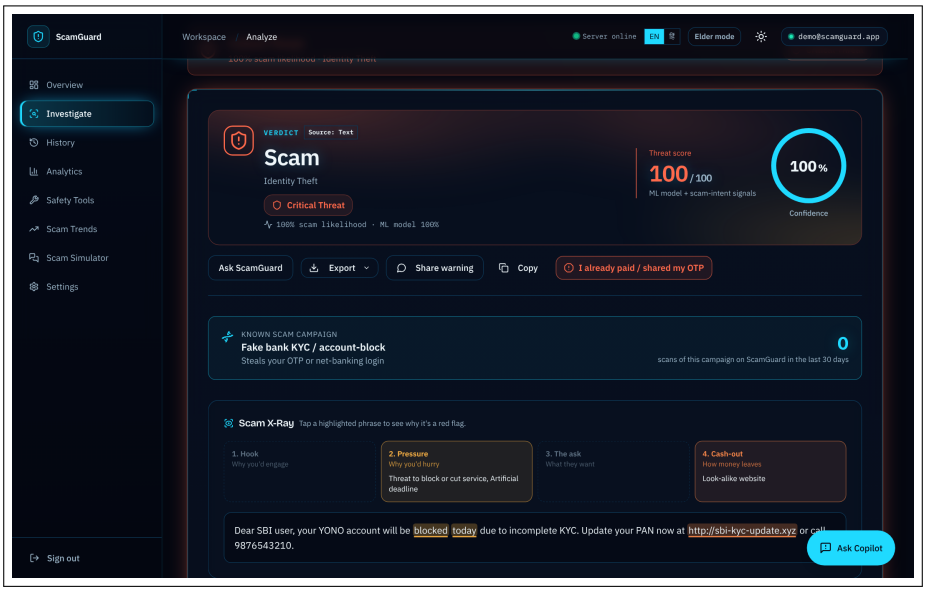


Figure 9.8: Scam X-Ray: each risky phrase placed in the hook, pressure, request and cash-out stages

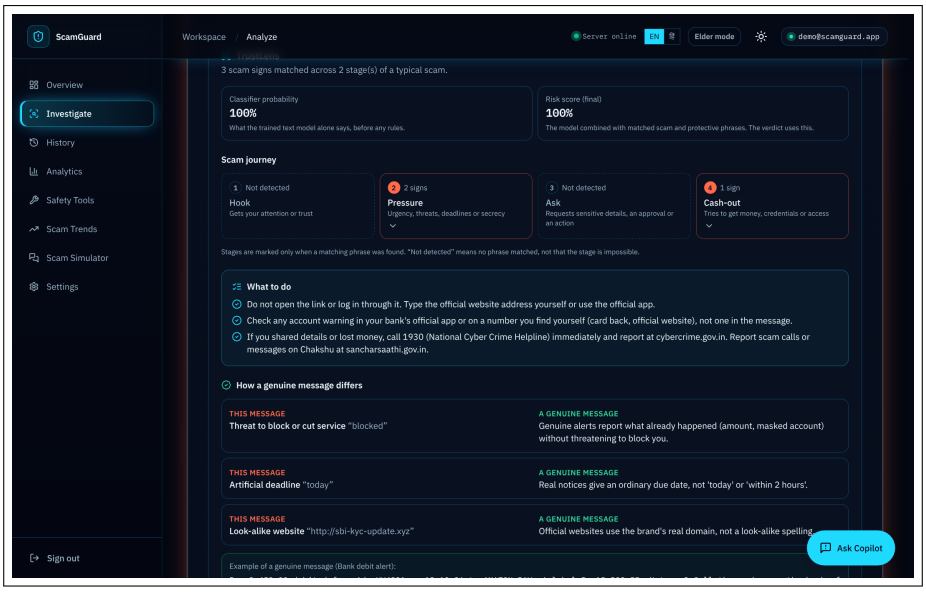


Figure 9.9: TrustLens: scam journey, recommended actions and how a genuine message differs

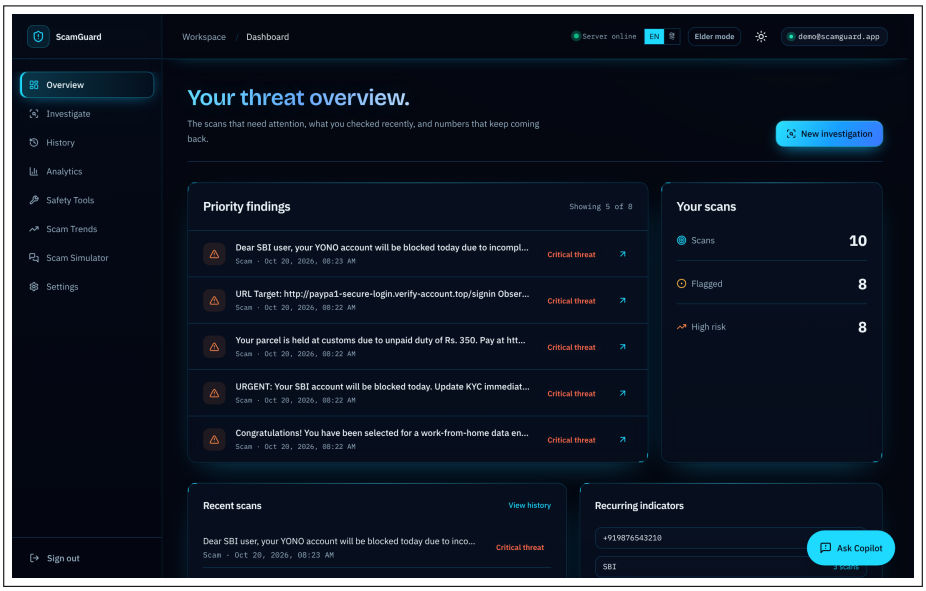


Figure 9.10: Dashboard with priority findings and recent scans



Figure 9.11: Analytics: verdict share and score trend

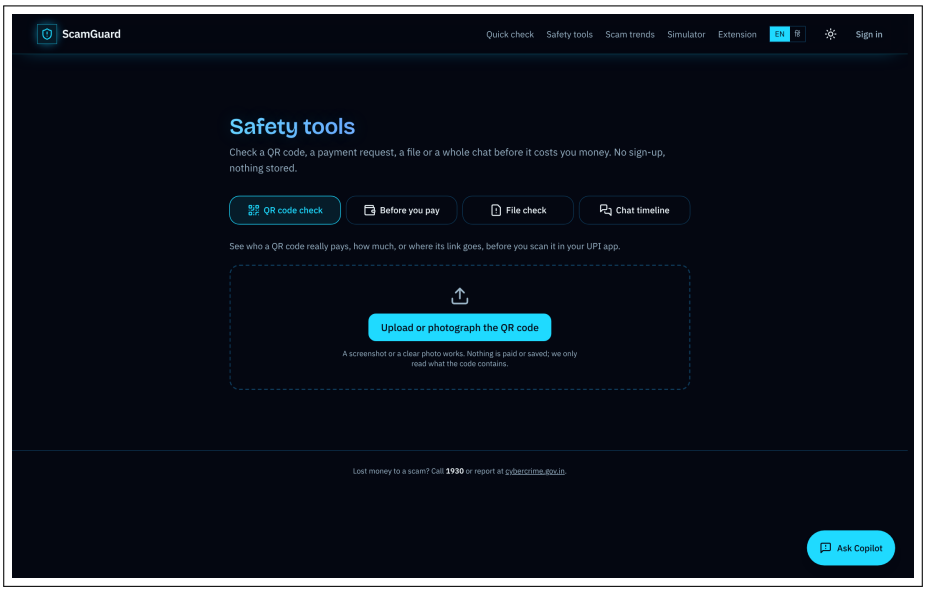


Figure 9.12: Safety tools: QR code check, before-you-pay check, file check and chat timeline

Chapter 10

Conclusion

Performance findings. On 1,874 real Indian messages from campaigns never seen in training, the deployed ScamGuard system reaches 97.1% accuracy, 98.2% precision, 97.2% recall and a 3.2% false-alarm rate, with a ROC-AUC of 0.995. It answers in 3.8 ms per message on a two-core server and needs only 1.6 MB for the model.

Comparison with the baseline. The common practice of treating every reported message as a scam flags 34.0% of genuine messages; the same model trained on audited labels flags 2.6%. A random split hides nearly half of the false alarms that a campaign-aware split reveals. Hand-written rules look strong on the developers' own messages but do not improve accuracy on independent data.

Contribution to the research gap. ScamGuard-IN-2026 provides what the literature lacked: an Indian scam-versus-genuine SMS dataset with a verified genuine class, an audit record for every reported-message group and a split that keeps each scam campaign on one side. The detector evaluated on it is deployed as an explainable client–server application.

Limitations. The 2025–26 scam types are represented in testing by only 12 advisory messages. One annotator made the manual decisions. Synthetic training data shares authors with the rules. Languages other than English and Hindi are not covered, and only simple obfuscation was tested.

Future work. Collect a larger real 2026 test set from team members, add a second annotator, extend to Kannada, Tamil and Telugu, test against paraphrased and adaptive scams, compare with small transformer models under the same split, and offer an Android share-to-check feature.

10.1 Mapping to course outcomes

Table 10.1 relates the work to the course outcomes of Major Project Phase-2 (23CS7PWPP2), and Table 10.2 gives the CO–PO–PSO mapping of the course.

Table 10.1: Course outcomes and where they are addressed

CO	Course outcome	Addressed in this work
CO1	Apply software engineering principles in planning, design and requirements within legal, societal and environmental constraints	Chapters 6–7: problem statement, architecture, sub-systems; licensed data and masking of personal data
CO2	Apply mathematics, science and engineering to implement the project and draw conclusions with integrity	Chapter 7: mathematical model; Chapter 9: reported results with confidence intervals
CO3	Monitor and manage schedules, resources and work assignments	Section 7.9: timeline, resources and cost
CO4	Test and defend the performance of the implemented project	Section 7.8 and Chapters 8–9: tests, metrics and validation
CO5	Work individually and as a team	Shared work on data, model, back end and front end
CO6	Communicate, present and prepare technical documents	This report, the research paper and the presentation

Table 10.2: CO–PO–PSO mapping of Major Project Phase-2

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1		2	2			2	2					2	2		
CO2	3			3	3			3						3	3
CO3											3				
CO4							1								1
CO5									1			1			
CO6										2		2			

References

[1] S. Agarwal, A. Papasavva, G. Suarez-Tangil, and M. Vasek, "Fishing for smishing: Understanding SMS phishing infrastructure and strategies by mining public user reports," in *Proc. ACM Internet Measurement Conference (IMC '25)*, Madison, WI, USA, 2025, doi: 10.1145/3730567.3764431.

[2] S. Agarwal, G. Suarez-Tangil, and M. Vasek, "An overview of 7726 user reports: Uncovering SMS scams and scammer strategies," in *Proc. Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2026.

[3] M. M. A. Pritom, S. M. Sanjari, M. Mia, A. M. Shibli, S. M. M. Hossain, M. Ismail, and S. Xu, "Short message service (SMS) phishing attacks and defenses: A systematic review," arXiv:2604.11429, 2026.

[4] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symposium on Document Engineering*, Mountain View, CA, USA, 2011, pp. 259–262.

[5] T. A. Almeida, J. M. Gómez Hidalgo, and T. P. Silva, "Towards SMS spam filtering: Results under a new dataset," *International Journal of Information Security Science*, vol. 2, no. 1, pp. 1–18, 2013.

[6] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep learning to filter SMS spam," *Future Generation Computer Systems*, vol. 102, pp. 524–533, 2020.

[7] S. Mishra and D. Soni, "Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis," *Future Generation Computer Systems*, 2020.

[8] G. Sonowal and K. S. Kuppusamy, "SmiDCA: An anti-smishing model with machine learning approach," *The Computer Journal*, vol. 61, no. 8, pp. 1143–1157, 2018, doi: 10.1093/comjnl/bxy039.

[9] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "Machine learning driven smishing detection framework for mobile security," arXiv:2412.09641, 2024.

[10] D. Timko and M. L. Rahman, "Smishing Dataset I: Phishing SMS dataset from Smishtank.com," in *Proc. 14th ACM Conference on Data and Application Security and Privacy (CODASPY '24)*, Porto, Portugal, 2024, doi: 10.1145/3626232.3653282.

[11] F. D. M. A. Ali, S. M. Saide, R. Sousa-Silva, and H. Lopes Cardoso, "MOZ-Smishing: A benchmark dataset for detecting mobile money frauds," in *Proc. 6th Workshop on African Natural Language Processing*, Vienna, Austria, 2025, pp. 158–166, doi: 10.18653/v1/2025.africanlp-1.23.

[12] M. Labonne and S. Moran, "Spam-T5: Benchmarking large language models for few-shot email spam detection," arXiv:2304.01238, 2023.

[13] M. Ahmadi, M. Khajavi, A. Varmaghani, A. Ala, K. Danesh, and D. Javaheri, "Leveraging large language models for cybersecurity: Enhancing SMS spam detection with robust and context-aware text classification," arXiv:2502.11014, 2025.

[14] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021, doi: 10.1613/jair.1.12125.

[15] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. 60th Annual Meeting of the ACL*, Dublin, Ireland, 2022, pp. 8424–8445, doi: 10.18653/v1/2022.acl-long.577.

[16] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 2023, doi: 10.1016/j.patter.2023.100804.

[17] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of SEQUENCES 1997*, Positano, Italy, 1997, doi: 10.1109/SEQUEN.1997.666900.

[18] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] S. Ramírez, "FastAPI documentation," 2026. [Online]. Available: https://fastapi.tiangolo.com

[21] Vercel, "Next.js documentation," 2026. [Online]. Available: https://nextjs.org/docs